National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

CS220: Database Systems

Class: BSAI – 2A

Lab 05: Aggregate Functions

CLO-2: Formulate SQL queries to retrieve information from a relational database.

CLO 3: Build SQL (DDL &DML) queries to retrieve information from a relational database.

Date: 26 Februrary 2026

Time: 09:45 – 11:55

Introduction

The GROUP BY clause can be used in a SELECT statement to collect data across multiple records and group the results by one or more columns.

Relational Algebra is a meta-language and forms underlying basis of SQL query language. It has six basic operators including: select, project, union, set difference, rename, and cross product. The operators take one or two relations as inputs and produce a new relation as a result.

Objectives

After completing this lab, you should be able to do the following:

- Identify the available group functions
- Describe the use of group functions
- Group data using the GROUP BY clause
- Include or exclude grouped rows by using the HAVING clause
- Extracting data from multiple tables

Tools/Software Requirement

MySQL workbench

Description

This lab further addresses functions. It focuses on obtaining summary information, such as averages, for groups of rows. It discusses how to group rows in a table into smaller sets and how to specify search criteria for groups of rows.

Instructions

Execute the company.sql script to create company schema first. After that, practice the given examples and all group functions of SQL. At the end, attempt the questions given as lab tasks in the manual.

NOTE:

Reference link for database creation:

https://justinsomnia.org/2009/04/the-emp-and-dept-tables-for-mysql/

SQL Group by Clause:

Lab Practice 1: using the SUM function

For example, you could also use the SUM function to return the department-id and the total salary (in the associated department).

```
SELECT deptno, SUM(sal) as "Total salary"
FROM emp
GROUP BY deptno;
```

OUTPUT:

Result Grid | Filter Rows:

deptno	Total salary
20	10875.00
30	9400.00
10	8750.00

Because you have listed one column in your SELECT statement that is not encapsulated in the SUM function, you must use a GROUP BY clause. The department field must, therefore, be listed in the GROUP BY section.

Lab Practice 2: using the COUNT function

For example, you could use the COUNT function to return the department-id and the number of employees (in the associated department) that make over $25,000 / year.

```
SELECT deptno, COUNT(*) as "Number of employees"
FROM emp
WHERE sal > 2500
GROUP BY deptno;
```

OUTPUT:

Result Grid | Filter Rows:

deptno	Number of employees
20	3
30	1
10	1

Lab Practice 3: using the MIN function

For example, you could also use the MIN function to return the department-id and the minimum salary in the department.

```
SELECT deptno, MIN(sal) as "Lowest salary"
FROM emp
GROUP BY deptno;
```

OUTPUT:

deptno	Lowest salary
20	800.00
30	950.00
10	1300.00

Lab Practice 4: using the MAX function

For example, you could also use the MAX function to return the department-id and the maximum salary in the department.

Formulate the query yourself.

SQL :

SELECT deptno,max(sal) as"Highest salary"

From emp

Group by deptno

OUTPUT:

deptno	Highest salary
20	3000.00
30	2850.00
10	5000.00

Lab Practice 5: using the SUM function

For example, you could also use the SUM function to return the department-id and the total sales (in the associated department). The HAVING clause will filter the results so that only departments with sales greater than $1000 will be returned.

```
SELECT deptno, SUM(sal) as "Total sales"
FROM emp
GROUP BY deptno
HAVING SUM(sal) > 1000;
```

OUTPUT:

```
SELECT deptno, SUM(sal) as "Total sales"
FROM emp
GROUP BY deptno
HAVING SUM(sal) > 10000;
```

Lab Practice 6: using the COUNT function

For example, you could use the COUNT function to return the name of the department and the number of employees (in the associated department) that make over $25,00 / year. The HAVING clause will filter the results so that only departments with more than 10 employees will be returned.

```
SELECT deptno, COUNT(*) as "Number of employees"
FROM emp
WHERE sal > 2500
GROUP BY deptno
HAVING COUNT(*) > 1;
```

OUTPUT:

deptno	Number of employees
20	3

Lab Practice 7: using the MAX function

For example, you could also use the MAX function to return the id of each department and the maximum salary in the department. The HAVING clause will return only those departments whose maximum salary is less than $50,000.

```
SELECT deptno, MAX(sal) as "Highest salary"
FROM emp
GROUP BY deptno
HAVING MAX(sal) < 5000;
```

OUTPUT:

deptno	Highest salary
20	3000.00
30	2850.00

ALIAS

SQL aliases are used to temporarily rename a table or a column heading.

SQL Aliases

SQL aliases are used to give a database table, or a column in a table, a temporary name.
Basically aliases are created to make column names more readable.

SQL Alias Syntax for Columns

SELECT *column_name* AS *alias_name*
FROM *table_name;*

SQL Alias Syntax for Tables

SELECT *column_name(s)*
FROM *table_name* AS *alias_name;*

Example:

SELECT first_name AS Customer

FROM sakila.customer;

AS keyword is optional, even if you remove it a column or table alias is formulated.

Lab TASKS:

Formulate SQL queries for following information needs and execute them in MySQL server.

- Find the highest, lowest, sum and average salary of all employees. Label the columns as Maximum, Minimum, Sum and Average respectively. Save your query.

CODE:

SELECT MAX(sal) as "Maximum",

MIN(sal) as "Minimum",

SUM(sal) as "Sum",

avg(sal) as "Average"

FROM emp

OUTPUT:

Maximum	Minimum	Sum	Average
5000.00	800.00	29025.00	2073.214286

- Find the highest, lowest, sum and average salary for each job type. Label the columns as Maximum, Minimum, Sum and Average respectively. Save your query.

CODE:

```
SELECT job, MAX(sal) as "Maximum",

MIN(sal) as "Minimum",

SUM(sal) as "Sum",

avg(sal) as "Average"

FROM emp

Group by job
```

OUTPUT:

job	Maximum	Minimum	Sum	Average
CLERK	1300.00	800.00	4150.00	1037.500000
SALESMAN	1600.00	1250.00	5600.00	1400.000000
MANAGER	2975.00	2450.00	8275.00	2758.333333
ANALYST	3000.00	3000.00	6000.00	3000.000000
PRESIDENT	5000.00	5000.00	5000.00	5000.000000

- Lists the number of employees in each job, sorted high to low.

CODE:

SELECT job,count(empno) as No_of_Employees

From emp

Group by job

Order by No_of_Employees DESC

OUTPUT:

Result Grid | Filter Rows:

job	No_of_Employees
CLERK	4
SALESMAN	4
MANAGER	3
ANALYST	2
PRESIDENT	1

- Display the number of distinct department values in the EMPLOYEES table.

CODE:

SELECT count(distinct deptno) AS "Distinct Department Values"

From emp

OUTPUT:

Result Grid | Filter Rows:

	Distinct Department Values
▶	3

- Determine the number of managers without listing them. Label the column as Number of Mangers.

CODE:

SELECT count(mgr) AS "No of Managers"

From emp

OUTPUT:

Result Grid | Filter Rows:

	No of Managers
▶	13

- Find the difference between highest and lowest salaries.

CODE:

SELECT MAX(sal) - MIN(sal) AS "Difference between Highest and Lowest Salaries"

From emp

OUTPUT:

Result Grid | Filter Rows:

Difference between Highest and Lowest Salaries
4200.00

- Formulate a query to display the manager number and the salary of the lowest-paid employee for that manager. Exclude any groups where the minimum salary is 4000 or less. Sort the output in descending order of salary.

CODE:

```
SELECT mgr,MIN(sal) AS Lowest_Salary

From emp

Where mgr IS NOT NULL

Group by mgr

Having MIN(sal) >4000

Order by Lowest_Salary Desc
```

OUTPUT:

Result Grid | Filter Rows:

mgr	Lowest_Salary

- Give the department names and their locations.

CODE:

```
SELECT dname AS "Department Names",

loc AS "Locations"

From dept
```

OUTPUT:

Department Names	Locations
ACCOUNTING	NEW YORK
RESEARCH	DALLAS
SALES	CHICAGO
OPERATIONS	BOSTON

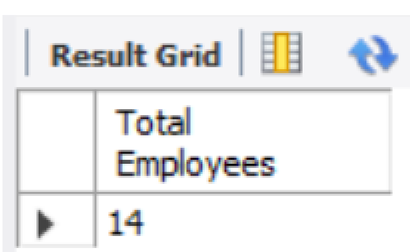
- Retrieve total no. of employees in the Company.

CODE:

```
SELECT count(empno) AS "Total Employees"

From emp
```

OUTPUT:



Total Employees
14

Deliverables: Save all queries and their results in the word document that you are executing, including examples and the questions. Relational algebra queries expressions saved in plain text file or word document. Upload this document to LMS. Late submissions will not be accepted.